	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 7

Lab Title: Queue and its application

Student Name: Muammad Waqas **Reg. No:** 240756

Objective:. Understand the concept of a queue and how it follows the FIFO (First In, First Out) . principle. Implement a queue using a linked list without using arrays or STL containers

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 7

Tittle : Queue and its Application

Name : Muammad waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 24 th October 24

Objective

The main objective of this program is to:

- Understand the concept of a queue and how it follows the FIFO (First In, First Out) principle.
- Implement a queue using a linked list without using arrays or STL containers.
- Learn how to use pointers to dynamically create and connect nodes.
- Practice basic enqueue (insert), dequeue (delete), and display operations using classes and objects in C++.

TASK 1

1. At a car service station, cars arrive one by one and are queued up for servicing.The car that arrives first is serviced first — this follows the FIFO **(First In, First Out)** rule.

====(CODE)====

```cpp
#include<iostream>
using namespace std;
class Car{
public:
int number;
string company;
string service;
Car(){}
Car(int n,string c,string s){
number=n;
company=c;
service=s;
}
};

template<typename T>
class Node{
public:
T data;
Node* next;
Node(T d){
data=d;
next=NULL;
}
};

template<typename T>
class Queue{
private:
Node<T>* front;
Node<T>* rear;
public:
Queue(){
front=NULL;
rear=NULL;
}
void enqueue(T val){
Node<T>* newNode=new Node<T>(val);
if(front==NULL){
front=newNode;
rear=newNode;
}
else{
rear->next=newNode;
rear=newNode;
}
cout<<"Car"<<val.number<<"added.\n";
}
void dequeue(){
if(front==NULL){
cout<<"QueueEmpty!\n";
return;
}
cout<<"Car"<<front->data.number<<"served.\n";
Node<T>* temp=front;
front=front->next;
```

====(CODE)====

```
57  delete temp;
58  if(front==NULL)rear=NULL;
59  }
60  void display(){
61  if(front==NULL){
62  cout<<"QueueEmpty\n";
63  return;
64  }
65  int n=1;
66  Node<T>* temp=front;
67  cout<<"Cars:\n";
68  while(temp!=NULL){
69      cout<<"====(CAR "<<(n++)<<")====\n";
70  cout<<"No:"<<temp->data.number<<endl;
71  cout<<"Company:"<<temp->data.company<<endl;
72  cout<<"Service:"<<temp->data.service<<"\n";
73  cout<<"----------------\n";
74  temp=temp->next;
75  }
76  }
77  bool isempty(){
78  return front==NULL;
79  }
80  };
81  int main(){
82  Queue<Car> q;
83  int ch,num;
```

```
84  string comp,serv;
85  while(true){
86  cout<<"=====(MainMenu)=====\n";
87  cout<<"1:-Add\n2:-Serve\n";
88  cout<<"3:-Show\n4:-Exit\nEnter:"
89  cin>>ch;
90  if(ch==1){
91  cout<<"EnterCarNumber:";
92  cin>>num;
93  cout<<"EnterCompany:";
94  cin>>comp;
95  cout<<"EnterServiceType:";
96  cin>>serv;
97  Car c(num,comp,serv);
98  q.enqueue(c);
99  }
100 else if(ch==2){
101 q.dequeue();
102 }
103 else if(ch==3){
104 q.display();
105 }
106 else if(ch==4){
107 return 0;
108 }
109 else{
110 cout<<"Invalid\n";
```

====(CODE)====	=====(OUTPUT)=====

```
84  string comp,serv;
85  while(true){
86  cout<<"=====(MainMenu)=====\n";
87  cout<<"1:-Add\n2:-Serve\n";
88  cout<<"3:-Show\n4:-Exit\nEnter:";
89  cin>>ch;
90  if(ch==1){
91  cout<<"EnterCarNumber:";
92  cin>>num;
93  cout<<"EnterCompany:";
94  cin>>comp;
95  cout<<"EnterServiceType:";
96  cin>>serv;
97  Car c(num,comp,serv);
98  q.enqueue(c);}
99  else if(ch==2){
100 q.dequeue();}
101 else if(ch==3){
102 q.display();}
103 else if(ch==4){
104 return 0;}
105 else{
106 cout<<"Invalid\n";}}}
107
```

Insertion and main menu

```
C:\Users\Nextsol Pvt Ltd\Downloads\LAB6\task1.exe
=====(MainMenu)=====
1:-Add
2:-Serve
3:-Show
4:-Exit
Enter:1
EnterCarNumber:10101
EnterCompany:TESLA
EnterServiceType:WASHING
Car10101added.
=====(MainMenu)=====
1:-Add
2:-Serve
3:-Show
4:-Exit
Enter:1
EnterCarNumber:20202
EnterCompany:HONDA
EnterServiceType:ENGINE_SERVICE
Car20202added.
=====(MainMenu)=====
1:-Add
2:-Serve
3:-Show
4:-Exit
Enter:1
EnterCarNumber:30303
EnterCompany:CIVIC
EnterServiceType:FULL_SERVICE
Car30303added.
```

=====(OUTPUT)=====

Display Function

```
====(CAR 1)====
No:10101
Company:TESLA
Service:WASHING
------------------
====(CAR 2)====
No:20202
Company:HONDA
Service:ENGINE_SERVICE
------------------
====(CAR 3)====
No:30303
Company:CIVIC
Service:FULL_SERVICE
------------------
```

DEQUEUE FUNCTION

```
=====(MainMenu)=====
1:-Add
2:-Serve
3:-Show
4:-Exit
Enter:2
Car10101served.
=====(MainMenu)=====
1:-Add
2:-Serve
3:-Show
4:-Exit
Enter:2
Car20202served.
=====(MainMenu)=====
1:-Add
2:-Serve
3:-Show
4:-Exit
Enter:3
Cars:
====(CAR 1)====
No:30303
Company:CIVIC
Service:FULL_SERVICE
------------------
```

\

TASK 2:

At an airport check-in counter, passengers line up to board in the order they arrive. However, Business Class passengers are always given priority over Economy Class passengers.

====(CODE)====

```
#include<iostream>
#include<string>
using namespace std;
class Passenger{
public:
string name;
string passport;
string seat;
Passenger(){}
Passenger(string n,string p,string s){
name=n;
passport=p;
seat=s;}
};
template<typename T>
class Node{
public:
T data;
Node* next;
Node(T d){
data=d;
next=NULL;
}};
template<typename T>
class Queue{
private:
Node<T>* front;
Node<T>* rear;
int count;
public:
Queue(){
front=NULL;
rear=NULL;
count=0;
}
void enqueue(T );
void dequeue();
void display();
bool isempty();
bool search(string pass);
void clear();
int size();
};
template<typename T>
void Queue<T>::enqueue(T val){
Node<T>* n=new Node<T>(val);
if(front==NULL){
front=n;
rear=n;
cout<<"First passenger added:\n";
}
else{
rear->next=n;
rear=n;
cout<<"Passenger added:\n";
}
```

====(CODE)====

```
57  count++;
58  cout<<"PassengerAdded.\n";
59  }
60  template<typename T>
61  void Queue<T>::dequeue(){
62  if(front==NULL){
63  cout<<"QueueEmpty!\n";
64  return;
65  }
66  cout<<"===(Served Pasenger:)===\n"<<endl;
67  cout<<front->data.name;
68  cout<<"("<<front->data.passport<<")\n";
69  Node<T>* t=front;
70  front=front->next;
71  delete t;
72  count--;
73  if(front==NULL)rear=NULL;
74  }
75  template<typename T>
76  void Queue<T>::display(){
77  if(front==NULL){
78  cout<<"QueueEmpty!\n";
79  return;
80  }
81  Node<T>* t=front;
82  cout<<"-----PassengerList-----\n";
83  int n=0;
84  while(t!=NULL){
85  cout<<"====(Passenger "<<(++n)<<")====\n";
86  cout<<"Name:"<<t->data.name<<endl;
87  cout<<"Passport:"<<t->data.passport<<endl;
88  cout<<"Seat:"<<t->data.seat<<endl;
89  t=t->next;
90  }
91  }
92  template<typename T>
93  bool Queue<T>::isempty(){
94  return front==NULL;
95  }
96  template<typename T>
97  int Queue<T>::size(){
98  return count;
99  }
100 template<typename T>
101 bool Queue<T>::search(string pass){
102 Node<T>* t=front;
103 while(t!=NULL){
104 if(t->data.passport==pass){
105 cout<<"Found:"<<t->data.name
106 <<" Seat:"<<t->data.seat<<"\n";
107 return true;
108 }
109 t=t->next;
110 }
111 cout<<"NotFound!\n";
```

====(CODE)====

```
112     return false;
113   }
114   template<typename T>
115   void Queue<T>::clear(){
116   while(!isempty()){
117   dequeue();}
118   cout<<"AllCleared.\n";
119   }
120
121   int main(){
122   Queue<Passenger> business;
123   Queue<Passenger> economy;
124   int ch;
125   string name,passport,seat,cls;
126   while(true){
127   cout<<"======Airport Check-In======\n";
128   cout<<"1:Add Passenger\n";
129   cout<<"2:Serve Passenger\n";
130   cout<<"3:Show All\n";
131   cout<<"4:Search\n";
132   cout<<"5:Show Counts\n";
133   cout<<"6:Clear All\n";
134   cout<<"7:Exit\n";
135   cout<<"Enter:";
136   cin>>ch;
137   if(ch==1){
138   cout<<"Name:";cin>>name;
139   cout<<"Passport:";cin>>passport;
140   cout<<"Seat:";cin>>seat;
141   cout<<"Class(B/E):";cin>>cls;
142
143   Passenger p(name,passport,seat);
144   if(cls=="B"||cls=="b"){
145   business.enqueue(p);
146   cout<<"AddedToBusiness.\n";
147   }
148   else{
149   economy.enqueue(p);
150   cout<<"AddedToEconomy.\n";
151   }}
152   else if(ch==2){
153   if(!business.isempty()){
154   business.dequeue();}
155   else if(!economy.isempty())
156   {
157   economy.dequeue();}
158   else cout<<"NoPassenger!\n";}
159   else if(ch==3){
160   cout<<"---BusinessClass---\n";
161   business.display();
162   cout<<"---EconomyClass---\n";
163   economy.display();}
```

====(CODE)====	====(OUTPUT)====

```
164  else if(ch==4){
165  cout<<"EnterPassportToSearch:";
166  cin>>passport;
167  if(!business.search(passport)){
168  economy.search(passport);
169  }}
170  else if(ch==5){
171  cout<<"BusinessCount:"<<business.size()<<endl;;
172  cout<<"EconomyCount:"<<economy.size()<<endl;
173  }
174  else if(ch==6){
175  business.clear();
176  economy.clear();
177  }
178  else if(ch==7){
179  return 0;}
180  else cout<<"Invalid!\n";
181  }}
```

DISPLAY

```
---BusinessClass---
-----PassengerList-----
====(Passenger 1)====
Name:WAQAS
Passport:QATAR
Seat:W1234Q
====(Passenger 2)====
Name:JUNAID
Passport:INDIA
Seat:1Q23CD
---EconomyClass---
-----PassengerList-----
====(Passenger 1)====
Name:Ali
Passport:Pakistan
Seat:A101G
====(Passenger 2)====
Name:UMAR
Passport:Q12VF
Seat:2W3E
```

====(OUTPUT)====

Main Menu

```
======Airport Check-In======
1:Add Passenger
2:Serve Passenger
3:Show All
4:Search
5:Show Counts
6:Clear All
7:Exit
Enter:
```

Insertion

```
======Airport Check-In======
1:Add Passenger
2:Serve Passenger
3:Show All
4:Search
5:Show Counts
6:Clear All
7:Exit
Enter:1
Name:Ali
Passport:Pakistan
Seat:A101G
Class(B/E):E
First passenger added:
PassengerAdded.
AddedToEconomy.
```

```
======Airport Check-In======
1:Add Passenger
2:Serve Passenger
3:Show All
4:Search
5:Show Counts
6:Clear All
7:Exit
Enter:2
===(Served Pasenger:)===

WAQAS(QATAR)
======Airport Check-In======
1:Add Passenger
2:Serve Passenger
3:Show All
4:Search
5:Show Counts
6:Clear All
7:Exit
Enter:2
===(Served Pasenger:)===

JUNAID(INDIA)
```

Conclusion:

- Queues help us manage data in the same order it arrives, just like people standing in a line.
- FIFO means the first person who comes is the first one to be served.
- Priority queues allow some people or tasks to be served earlier based on importance.
- Business Class passengers get priority, so they are served before Economy passengers even if they arrive later.
- Using queues makes the system organized and avoids confusion during check-in or service.
- Linked lists help queues grow and shrink easily without wasting memory.
- Priority queues are useful in real-life systems like airports, hospitals, and computer scheduling because they handle urgent tasks first.